

Uma arquitetura de conjunto de instruções para realização de métodos numéricos

Walter Bolitto Carvalho
Universidade Federal do ABC
walter.carvalho@ufabc.edu.br

Resumo — O presente trabalho discute a construção de uma arquitetura de conjunto de instruções para a resolução de problemas matemáticos, envolvendo métodos numéricos. A arquitetura apresentava um número limitado de registradores, memória interna e instruções únicas, gerando um desafio na resolução apresentados. No decorrer dos testes realizados com três problemas matemáticos, foi observado resultados satisfatórios, mostrando o potencial de realizar cálculos complexos com sistemas minimalistas.

Palavras Chave — arquitetura de computadores, arquitetura de conjunto de instruções.

I. INTRODUÇÃO

A área de Arquitetura de computadores condensa uma série de conhecimentos interdisciplinares, abrangendo conceitos matemáticos, físicos, químicos e da engenharia. Além disso, nota-se a importância da compreensão do desenvolvimento histórico da área, que justifica escolhas de engenharia em organização de peças, hierarquia de elementos e como diferentes níveis de abstração foram estruturados enquanto linguagem.

No início da década de 1980, na busca pelo aumento do desempenho, dedicou-se uma maior atenção ao conceito de arquitetura de conjunto de instruções (ISA – Instruction Set Architecture), onde houve uma atenção também a questão da compatibilidade de arquiteturas em um contexto de paralelismo de *cores* [1].

Em uma ISA, o conjunto de instruções pode ser definido como o “conjunto de palavras que compõem o idioma inteligível ao processador e outras partes da organização do computador. (...) Este conjunto é rigorosamente controlado pelo Sistema Operacional, ou seja, um software que deseja fazer uso de tais instruções precisa de permissão especial. (Netto, 2022, p. 25) [2].

ISAs apresentam alguns elementos importantes para seu funcionamento, entre eles: (a) endereçamento de memória, usando bytes para acessar e realizar operações; (b) categorias de operações, incluindo geralmente operações de transferências de dados, operações lógico-aritméticas, controle e de ponto flutuante; (c) instrução de controle de fluxo, para realizar o sequenciamento de ações, pulsos não-condicionais, *loops*, retornos, etc. [3]

Foi elaborada uma ISA capaz de resolver determinados problemas matemáticos, sendo eles: cálculo de seno, cálculo de cosseno e cálculo da raiz n -ésima. A ISA proposta, bem como a implementação da arquitetura com os diferentes

métodos numéricos abordados pode ser acessada por meio do link do Github: <https://github.com/walterbolitto/adcfabc/tree/main/isa>

II. ARQUITETURA E ISA

Foi utilizada a linguagem Python para criar um programa que fosse capaz de simular o comportamento de uma ISA, com o objetivo de aplicar os conhecimentos desenvolvidos na disciplina ‘TCCM20120253 - Arquiteturas de Computadores – 2025.Q3’ sobre arquiteturas de conjuntos de instruções, de forma que as instruções definidas foram baseadas nas operações matemáticas básicas, além de *whiles* e *ifs* existentes na Linguagem Python para operar a instrução Jump e o funcionamento do contador de programa.

A ISA proposta estabeleceu dez instruções únicas para o desenvolvimento dos três programas que fazem uso de métodos numéricos, simulando o padrão das principais instruções do Assembly, sendo elas: soma, subtração, multiplicação, divisão, carregamento de dados para registradores, salvamento de dados na memória interna, comparar se menor que (SLT - Set on less Than), comparar se maior que (SGT – Set on greater than), desvie se for igual a (BEQ – Branch on Equal) e Pulo (Jump).

As instruções únicas foram estabelecidas como funções do Python de forma que também simulassem o opcode existente no Mars Mips¹ para melhor demonstrar como a linguagem Assembly é comumente representada:

```
####Instrução Soma (semelhante para sub, div e multiplicação)
def add(a, b):
```

```
    return a + b
```

```
####Exemplo da instrução add
```

```
    reg["$s0"] = add(reg["$s0"],360)
```

```
####Instrução Salvar (semelhante a carregar)
```

```
def sw(a, b):
```

```
    ad_100100000[a] = b
```

```
#### Exemplo da instrução Sw
```

```
    sw(0,reg["$s1"])
```

¹MARS MIPS Assembler and Runtime Simulator - Computer Science Department - Missouri State. Disponível em: <https://computerscience.missouristate.edu/mars-mips-simulator.htm>. Acesso em 05/12/2025.

```
###SGT (semelhante a instrução SLT)
```

```
def sgt(a, numsgt, c):
```

```
    if a > numsgt:
```

```
        reg["$s6"] = c
```

```
###BEQ
```

```
def beq(a,b,label):
```

```
    if a == b:
```

```
        reg["$jump"] = label
```

```
### Exemplo de aplicação
```

```
    sgt(reg["$s2"],3, 11)
```

```
    beq(reg["$s6"],11,11)
```

Também foi estabelecido o uso de oito registradores: \$s0, \$s1, \$s2, \$s3, \$s4, \$s5, \$s6 e \$jump, bem como 64 palavras de 4 bytes de memória interna da arquitetura, organizadas em oito endereços, estruturados de forma que também simule a estrutura encontrada no programa Mars Mips, apresentada da seguinte forma: ad_100100000 = { 0 : 0, 4 : 3, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }; ad_100100020 = { 0 : 0, 4 : 0, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }; ad_100100040 = { 0 : 0, 4 : 0, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }; ad_100100060 = { 0 : 0, 4 : 0, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }; ad_100100080 = { 0 : 0, 4 : 0, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }; ad_1001000a0 = { 0 : 0, 4 : 0, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }; ad_1001000c0 = { 0 : 0, 4 : 0, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }; e ad_1001000e0 = { 0 : 0, 4 : 0, 8 : 0, 12 : 0, 16 : 0, 20 : 0, 24 : 0, 28 : 0 }.

III. IMPLEMENTAÇÃO DO EMULADOR

O emulador foi implementado na ferramenta Jupyter Notebook². A Figura 1 apresenta brevemente o sequenciamento da arquitetura do emulador.



² JupyterLite. Disponível em: <https://jupyter.org/try-jupyter/lab/>. Acesso em 04/12/2025.

Fig 1. Representação da arquitetura do emulador

Considerando o limitado número de registradores, foi necessário uma atenção ao uso dos mesmos no emulador para realização das operações, de forma que foram delimitados alguns usos. Foi estabelecido um registrador \$jump ao invés de \$s7 para estabelecer o sequenciamento das operações a serem realizadas, bem como funcionarem como marcadores para eventuais *jumps*. Considerando o potencial das funções SGT e SLT para realizar a operação *jump*, foi definido que o registrador \$s6 serviria para substituir o número armazenado quando as instruções de comparações fossem realizadas. Os outros registradores não apresentavam função fixa.

IV. MÉTODOS NUMÉRICOS

Em relação ao primeiro programa, cálculo de senos, foi utilizado o método de Newton-Raphson, que se utiliza de uma série de Taylor para realizar a aproximação do valor desejado. Foram observados os seguintes desafios para o programa: (a) em ângulos maiores de 130°, os resultados obtidos eram pouco satisfatórios; (b) assim como o segundo programa, o pequeno número de registradores exigiu um grande controle dos registradores e amplo uso de operações de incremento, sendo também necessário o uso da memória interna para completude do programa proposto; (c) foi necessário estabelecer um caso particular para 0° devido a operações de divisões presentes na série de Taylor. [4]

Em relação ao segundo programa, cálculo de cossenos, também foi utilizado o método de Newton-Raphson, que apresenta uma pequena variação na série de Taylor. O programa apresentou desafios semelhantes ao cálculo de senos.

Em relação ao terceiro programa, raiz n-ésima, o principal desafio foi encontrar métodos de solucionamento que fossem viáveis de realizar com a ISA proposta, que realiza apenas operações básicas, sendo utilizado o Método de Newton, conhecido também como método das tangentes, para obtenção da raiz n-ésima.³

V. RESULTADOS

Em relação ao primeiro programa, considerando os desafios listados na seção anterior, foram realizadas operações numéricas para gerar resultados mais satisfatórios, já que alguns cálculos chegavam a valores maiores do que 1 ou menores do que menos 1. A primeira escolha definida foi a obtenção de ângulos coterminais, considerando que não poderia ser utilizada a função 'resto da divisão', foi feito um loop para subtrair 360 do valor original até ser alcançado um valor satisfatório.

Em seguida, considerando que valores acima de 130° apresentavam resultados indesejados, buscou-se os ângulos suplementares do primeiro e do quarto quadrante para obtenção de resultados mais interessantes. A Fig. 2 apresenta um panorama com testes realizados com o

³RPM 04 - A raiz n-ésima pelo método das aproximações sucessivas. Disponível em: <https://rpm.org.br/cdrpm/4/7.htm>. Acesso em 01/12/2025.

programa, bem como os valores obtidos na plataforma Wolfram Alpha⁴.

Escolhas semelhantes foram observadas no programa para descobrir valores de cossenos, diferindo do programa anterior com base na identificação do quadrante no qual o ângulo se manifesta, a fim de definir se o número é positivo ou negativo. A Fig. 3 apresenta um paralelo com testes do problemas relacionado aos resultados obtidos no Wolfram Alpha.



Fig 2. Comparativo de resultados obtidos para obtenção de seno com o Wolfram Alpha

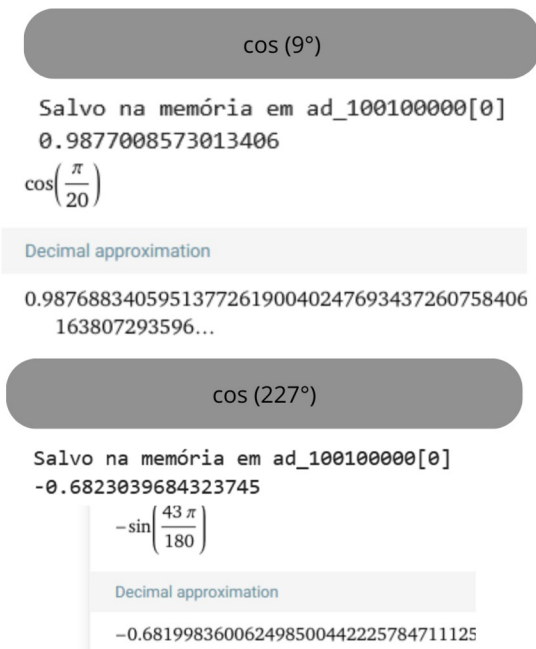


Fig 3. Comparativo de resultados obtidos para obtenção de cosseno com o Wolfram Alpha

Com os resultados obtidos nos programas de seno e cosseno, nota-se uma precisão de até duas casas decimais em relação ao identificado na calculadora digital, sendo considerado resultados satisfatórios.

No contexto do algoritmo para obtenção da raiz n-ésima, a implementação foi mais simples de ser realizada em relação à obtenção do seno e cosseno. Para aprimorar os resultados obtidos, foram realizadas oito iterações para maior precisão dos resultados obtidos. A Fig. 4 apresenta um comparativo com a calculadora digital Wolfram Alpha.

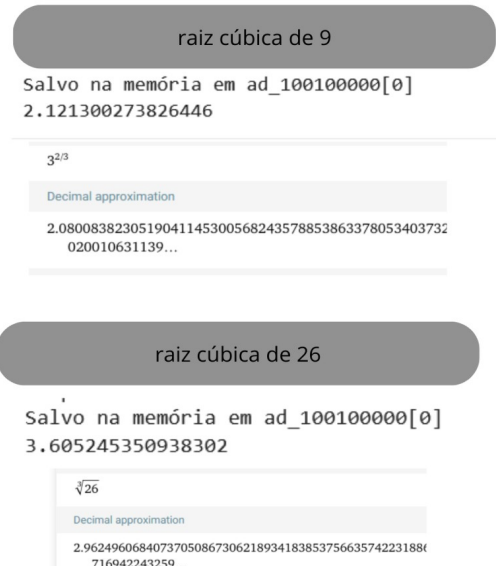


Fig 4. Comparativo de resultados obtidos para obtenção da raiz n-ésima com o Wolfram Alpha

Pelos resultados obtidos no programa para obtenção da raiz, nota-se que o método se aproxima do valor esperado, mas não oferece a precisão de uma casa decimal para valores grandes, podendo ser inadequado para vários contextos possíveis. Desta forma, sugere-se a utilização de outro método numérico para a obtenção de resultados mais satisfatórios.

VI. CONCLUSÃO

O algoritmo proposto atingiu parcialmente o objetivo proposto de 'apresentar uma arquitetura de conjunto de instruções capaz de resolver problemas matemáticos por métodos numéricos' como obtenção de valores de seno, cosseno e raiz n-ésima de dado valor. Para seu desenvolvimento, foram utilizados métodos diversos, fazendo uso de iterações e da série de Taylor.

O método numérico de Newton-Raphson mostrou-se adequado para obtenção de senos e cossenos, enquanto o método de Newton mostrou-se pouco eficiente para obtenção da raiz n-ésima.

⁴ Wolfram | Alpha: Computational Intelligence. Disponível em: <https://www.wolframalpha.com/>. Acesso em 04/12/2025.

O algoritmo também possibilitou um aprofundamento nos conhecimentos da área de arquitetura de computadores, bem como de métodos numéricos encontrados na literatura.

REFERÊNCIAS

- [1] W. Stallings. “Arquitetura e organização de computadores”. Pearson, 2017.
- [2] E. B. Netto. ‘Arquitetura de computadores: a visão do software’. 2022.
- [3] J. L. Hennessy, D. A. Patterson. ‘Computer architecture: a quantitative approach’. Elsevier, 2011.
- [4] S. Akram, Q. U. Ann. ‘Newton raphson method’. International Journal of Scientific & Engineering Research, 6(7), 1748-1752. 2015